Solution: Count Unique Characters of All Substrings of a Given String

Problem Statement

Given a string `s`, return the sum of `unique` characters in all possible substrings of a given string. You also need to count the unique characters of `repeating` substrings.

Examples

- **Example 1**:
  - **Input**: "aac"
  - **Expected Output**: 8
  - **Justification**: The substrings are "a", "a", "c", "aa", "ac", "aac". Their unique characters count are 1, 1, 1, 0, 2, 1 respectively. Summing up the unique characters: 1 + 1 + 1 + 0 (from "aa") + 2 (from "ac") + 1 (from "aac") = 6.
- **Example 2**:
  - **Input**: "aaa"
  - **Expected Output**: 6
  - **Justification**: The substrings are "a", "a", "a", "aa", "aa", and "aaa". Their unique characters count are 1, 1, 1, 0, 0, 0 respectively. So, there are total `3` unique characters in all substrings.
- **Example 3**:
  - **Input**: "abca"
  - **Expected Output**: 18
  - **Justification**: The substrings and their unique character counts are as follows: "a" (1), "b" (1), "c" (1), "a" (1), "ab" (2), "bc" (2), "ca" (2), "abc" (3), "bca" (3), "abca" (2). The sum of unique characters is **18**.

Solution

The solution to the problem of counting unique characters of all substrings of a given string employs an efficient strategy that leverages the positioning of characters within the string. Rather than enumerating all possible substrings, which would be computationally expensive, the approach calculates the contribution of each character based on its occurrence positions. This is done by maintaining a record of the last two positions where each character was found. By considering the distance between these occurrences and their positions relative to the string's start and end, we can determine how many unique substrings each character contributes to without direct enumeration. This method significantly reduces the computational complexity, making it feasible to handle strings of considerable length.

The algorithm iterates through the string once, updating the contribution of each character as it proceeds. This is achieved by maintaining a two-dimensional array where each row corresponds to a character of the alphabet and contains the last two positions of that character's occurrence. By calculating the contribution of characters based on the differences between their last positions and their position in the current iteration, the algorithm efficiently aggregates the total count of unique characters across all substrings. The final step involves adjusting this count to include contributions from substrings that end with each character, ensuring that all unique characters are accounted for.

Step-by-Step Algorithm

1. Initialize a 2D array `index` with dimensions 26x2 to store the last two positions for each alphabet character, initializing all values to -1.
2. Initialize variables `res` (for the result) and `mod` (for modular arithmetic to prevent overflow) with appropriate values.
3. Iterate through each character of the input string, performing the following steps:
   - Convert the current character to an index corresponding to its position in the alphabet (0-25).
   - Update `res` by adding the product of the difference between the current position and the character's last occurrence, and the difference between its last two occurrences. Apply modulo operation to prevent overflow.
   - Update the `index` array for the current character to reflect its new last occurrence and the previous last occurrence.
4. After iterating through the string, make a final pass over the alphabet characters to adjust `res` by accounting for substrings that end with each character. This involves adding the contribution of substrings that extend to the end of the string from each character's last occurrence.
5. Return `res` as the total count of unique characters across all substrings.

Algorithm Walkthrough

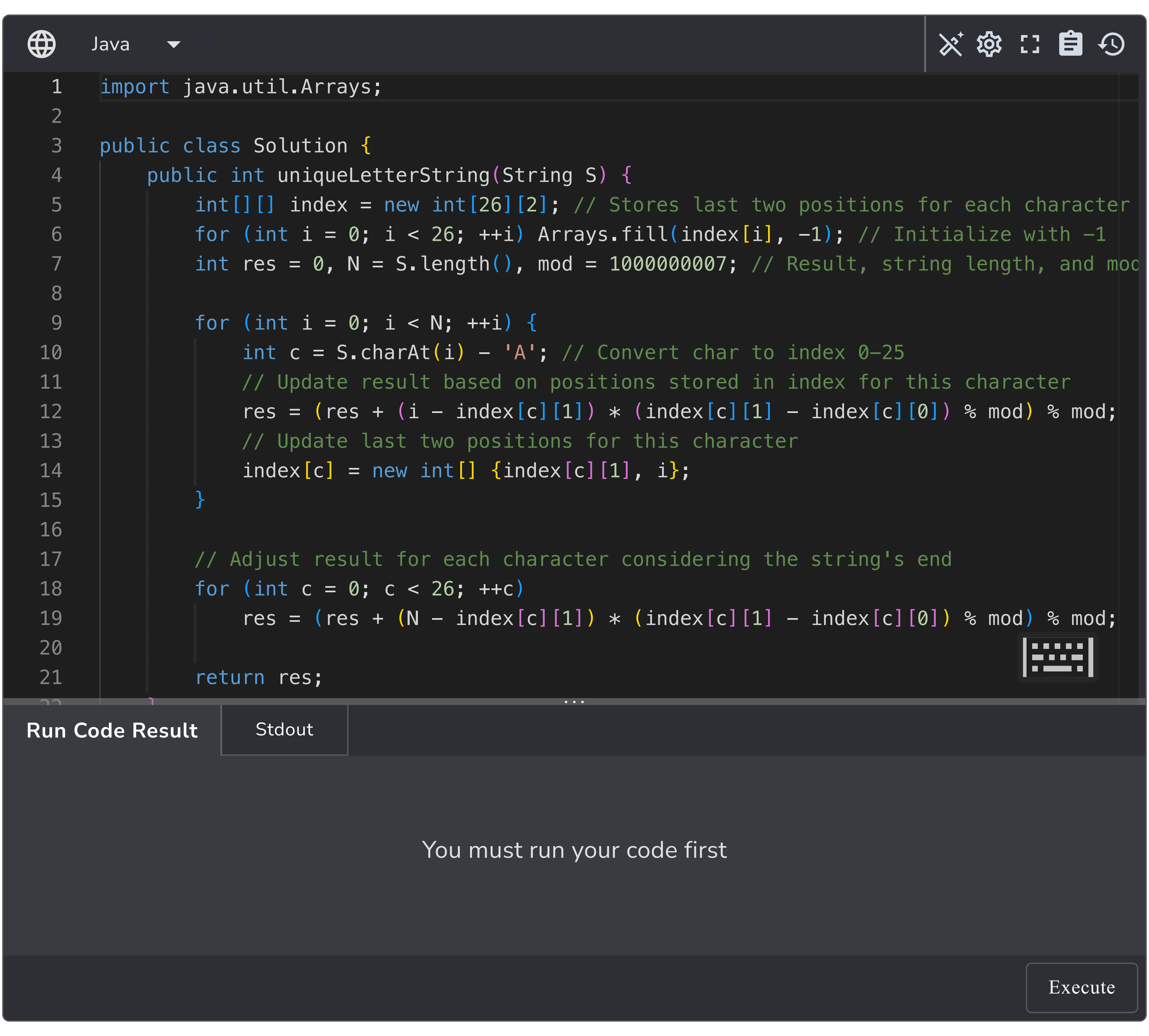
Let's consider the input: `s` = "ABCA"

1. Initialize `index` as a 2D array with all values set to -1, `res` to 0, `N` to 4 (length of "ABCA"), and `mod` to (10^9 + 7).
2. For character 'A' at position 0:
   - 'A' converts to index 0. Its last positions update from `[-1, -1]` to `[-1, 0]`.
   - `res` updates by adding `(0 - (-1)) * ((-1) - (-1)) % mod` which simplifies to 0.
3. For character 'B' at position 1:
   - 'B' converts to index 1. Its last positions update from `[-1, -1]` to `[-1, 1]`.
   - `res` updates by adding `(1 - (-1)) * ((-1) - (-1)) % mod` which simplifies to 0.
4. For character 'C' at position 2:
   - 'C' converts to index 2. Its last positions update from `[-1, -1]` to `[-1, 2]`.
   - `res` updates by adding `(2 - (-1)) * ((-1) - (-1)) % mod` which simplifies to 0.
5. For character 'A' at position 3:
   - 'A' converts to index 0 again. Its last positions update from `[-1, 0]` to `[0, 3]`.
   - `res` updates by adding `(3 - 0) * (0 - (-1)) % mod` which simplifies to 3.
6. Final adjustment for 'A':

- **For 'A'**: The adjustment involves calculating the contribution from the last occurrence (index 3) to the end of the string. The calculation `(4 - 3) * (3 - 0) % mod` adds to `res`. So, `res` becomes `6`.
- **For 'B'**: The adjustment for 'B' involves the substring ending with 'B', calculated as `(4 - 1) * (1 - (-1)) % mod`, contributing to `res`. So, `res` updates to `12`.
- **For 'C'**: Similarly, the adjustment for 'C' is `(4 - 2) * (2 - (-1)) % mod`, contributing to `res`. So, `es` updates to `18`.

8. The final value of `res` is 18.

Code



```java
import java.util.Arrays;

public class Solution {
    public int uniqueLetterString(String S) {
        int[][] index = new int[26][2]; // Stores last two positions for each character
        for (int i = 0; i < 26; ++i) Arrays.fill(index[i], -1); // Initialize with -1
        int res = 0, N = S.length(), mod = 1000000007; // Result, string length, and mo

        for (int i = 0; i < N; ++i) {
            int c = S.charAt(i) - 'A'; // Convert char to index 0-25
            // Update result based on positions stored in index for this character
            res = (res + (i - index[c][1]) * (index[c][1] - index[c][0]) % mod) % mod;
            // Update last two positions for this character
            index[c] = new int[] {index[c][1], i};
        }

        // Adjust result for each character considering the string's end
        for (int c = 0; c < 26; ++c)
            res = (res + (N - index[c][1]) * (index[c][1] - index[c][0]) % mod) % mod;

        return res;
```

Complexity Analysis

Time Complexity: $O(N)$

The code iterates through each character of the string exactly once to calculate its contribution to the result. This leads to a linear iteration with $O(N)$ complexity.

Space Complexity: $O(1)$

- The space used by the `index` array is constant, as it always contains 26 elements, each of which has 2 integers. This size does not change regardless of the input string's length, resulting in $O(1)$ space complexity.

← Previous
Count Unique Characters Of All Substrings Of A Given String (Hard)

Next →
Word Ladder (Hard)

☐ Mark as Completed